

Отчёт по требованиям к системе VBank

1. Назначение системы

VBank — внутренняя виртуальная банковская система для учёта денежных средств пользователей, ведения счетов, выполнения внутренних операций, обработки ручных заявок и ведения аудита значимых действий. Система работает в замкнутом контуре без интеграции с внешними банками, платёжными шлюзами и реальными внешними расчётами.

2. Роли пользователей

2.1. Клиент (`Client`)

Пользователь, который работает только со своими счетами, операциями и заявками. Клиент не имеет доступа к чужим объектам и административным функциям.

2.2. Оператор (`Operator`)

Пользователь, который обрабатывает заявки, управляет статусами пользователей и выполняет служебные действия в рамках своих полномочий. Оператор принимает итоговое решение по заявкам.

2.3. Администратор (`Admin`)

Пользователь, который управляет справочниками, режимами работы системы и аудитом.

2.4. Общее правило ролей

У пользователя в каждый момент времени ровно одна активная роль. Заблокированный пользователь не может входить в систему и инициировать новые действия.

3. Функциональные требования

3.1. Аутентификация и сессии

1. Система должна регистрировать пользователя напрямую при режиме ``registration_mode = auto``.
2. Система должна создавать заявку на регистрацию при режиме ``registration_mode = manual``.
3. Система должна выполнять вход по ``username``, ``email`` или ``phone_number``.
4. Система должна выдавать ``access token`` и ``refresh token``.
5. Система должна поддерживать обновление ``refresh token``.
6. Система должна поддерживать отзыв ``refresh token``.

3.2. Управление пользователями

7. Система должна хранить пользователя с одной активной ролью доступа.
8. Система должна обеспечивать уникальность ``email``, ``username`` и ``phone_number``.
9. Система должна позволять оператору просматривать пользователей.
10. Система должна позволять оператору блокировать и разблокировать пользователей.
11. Система должна позволять администратору изменять роль пользователя.
12. Система должна запрещать вход и новые действия для пользователя в статусе ``Blocked``.

3.3. Управление счетами

13. Система должна открывать счёт в активной валюте и существующем типе счёта.
14. Система должна открывать счёт напрямую при ``account_opening_mode = auto``.
15. Система должна создавать заявку на открытие счёта при ``account_opening_mode = manual``.
16. Система должна генерировать уникальный 20-значный ``account_number``.
17. Система должна обеспечивать уникальность сочетания ``(user_id, currency_id, account_type_code)``.
18. Система должна позволять закрыть счёт только при ``balance = 0``.
19. Система должна запрещать новые операции по счёту в статусе ``Closed``.

20. Система должна учитывать правило отрицательного баланса только для типов счетов, у которых это разрешено.

3.4. Финансовые операции

21. Система должна поддерживать операции `Transfer`, `Deposit`, `Withdraw`, `Compensation`.

22. Система должна выполнять финансовую операцию атомарно: целиком или не выполнять её вовсе.

23. Система должна использовать хранимый баланс счёта как рабочее состояние.

24. Система должна создавать проводки только для успешной операции.

25. Система должна запрещать отмену проведённой операции.

26. Система должна использовать `Compensation` для исправления ранее проведённой операции.

27. Система должна требовать, чтобы сумма операции была больше нуля.

28. Система должна требовать совпадения валюты операции и затронутых счетов.

3.5. Переводы между счетами

29. Система должна выполнять перевод напрямую при `internal\_transfer\_mode = enabled`.

30. Система должна создавать заявку на перевод при `internal\_transfer\_mode = manual`.

31. Система должна запрещать перевод при `internal\_transfer\_mode = disabled`.

32. Система должна запрещать перевод на тот же счёт.

33. Система должна проверять право инициатора распоряжаться счётом-источником, если инициатором не является оператор.

34. Система должна создавать две проводки при успешном переводе.

35. Система должна разрешать уход в отрицательный баланс только для перевода и только в пределах допустимого лимита.

3.6. Пополнение и вывод

- 36. Система должна обрабатывать пополнение и вывод только через заявку.
- 37. Система должна создавать финансовую операцию только после одобрения заявки.
- 38. Система должна создавать одну проводку для `Deposit`.
- 39. Система должна создавать одну проводку для `Withdraw`.
- 40. Система должна запрещать `Withdraw`, если итоговый баланс становится отрицательным.
- 41. Система должна отключать пополнение и вывод при `cash\_in\_out\_mode = disabled`.

3.7. Заявки

- 42. Система должна поддерживать типы заявок `UserRegistration`, `AccountOpening`, `Deposit`, `Withdraw`, `Transfer`.
- 43. Система должна использовать статусы `PendingApproval`, `Approved`, `Rejected`.
- 44. Система должна позволять оператору одобрять или отклонять заявку с указанием `ReasonCode`.
- 45. Система должна создавать результат предметной области только после `Approved`.
- 46. Система не должна создавать результат после `Rejected`.
- 47. Система должна валидировать `payload` заявки в зависимости от `request\_type`.
- 48. Система должна использовать заявку только для сценариев, переведённых в ручной режим.

3.8. Справочники и режимы работы

- 49. Система должна хранить справочники `Currency`, `AccountType`, `ReasonCode`.
- 50. Система должна запрещать физическое удаление справочных записей.
- 51. Система должна хранить только фиксированный набор ключей `SystemSetting`.
- 52. Система должна применять режимы работы системы к пользовательским сценариям без перезапуска приложения.
- 53. Система должна поддерживать режимы `registration\_mode`, `account\_opening\_mode`, `internal\_transfer\_mode`, `cash\_in\_out\_mode`.

3.9. Аудит и история

- 54. Система должна показывать клиенту только его собственные счета, операции и заявки.
- 55. Система должна показывать оператору историю пользователей и очередь заявок.
- 56. Система должна показывать администратору журнал аудита.
- 57. Система должна фиксировать в аудите каждую значимую попытку и результат.
- 58. Система должна обеспечивать корреляцию действий с `request\_id`.

3.10. API

- 59. Система должна предоставлять API с базовым путём `/api/v1`.
- 60. Система должна возвращать ошибки в формате `{code, message, details, request\_id}`.
- 61. Система должна требовать `Idempotency-Key` для всех изменяющих запросов.
- 62. Система должна управлять доступом к операциям по роли пользователя.

3.11. Интерфейс

- 63. Система должна предоставлять клиенту вход, регистрацию, список счетов, карточку счёта, создание счёта, перевод, создание заявок, историю операций и заявок.
- 64. Система должна предоставлять оператору список пользователей, карточку пользователя, очередь заявок, карточку заявки с решением, историю операций и заявок пользователей.
- 65. Система должна предоставлять администратору управление справочниками, управление режимами работы системы и просмотр аудита.

4. Нефункциональные требования

4.1. Производительность

1. Денежная операция должна укладываться в обычный интерактивный отклик.
2. Списки должны поддерживать пагинацию.
3. Денежные сценарии должны выполняться в одной транзакции БД без частичной фиксации результата.
4. Конкурентный доступ к затрагиваемым счетам должен контролироваться блокировкой строк, чтобы избежать расхождения баланса при параллельных операциях.

4.2. Надёжность

1. Система должна использовать PostgreSQL как основное хранилище данных.
2. Денежные операции должны выполняться в транзакции БД.
3. Система должна обеспечивать атомарность изменения баланса и создания проводок.
4. Проведённая операция не должна отменяться; исправление должно выполняться только компенсирующей операцией.
5. Система должна поддерживать идемпотентность изменяющих запросов.
6. Данные предметной области не должны удаляться физически; вместо этого должен использоваться статус или отзыв.
7. Миграции БД должны быть только вперёд.

4.3. Безопасность

1. Передача данных должна выполняться только по TLS.
2. Пароли должны храниться в виде безопасного хэша.
3. `refresh token` должен быть отзывным.
4. Секреты не должны храниться в репозитории.
5. Доступ к операциям должен управляться по роли пользователя.
6. Клиент должен иметь доступ только к собственным объектам.
7. Заблокированный пользователь не должен иметь возможность входа и инициации новых действий.

4.4. UX

1. Интерфейс должен быть разделён по задачам ролей: клиент, оператор, администратор.
2. Пользователь должен видеть только релевантные для своей роли сценарии и данные.
3. Ошибки API должны возвращаться в единообразном формате.
4. История операций, заявок и аудита должна быть доступна в явном виде для соответствующей роли.
5. Навигация и общие элементы интерфейса должны выноситься в общий слой `shared`, без переноса туда доменной логики.
6. Для списков пользовательских объектов должна использоваться пагинация.
7. Критичные действия ручной обработки должны выполняться через карточку заявки с явным решением оператора.

5. Ограничения

1. Система является внутренней виртуальной банковской системой и не работает с внешними банками, платёжными шлюзами и внешними расчётами.
2. Валютная конвертация не входит в рамки системы.
3. KYC, AML и комплаенс-процедуры не входят в рамки системы.
4. Отмена проведённой операции не поддерживается; допускается только компенсация.
5. Сложный многоэтапный процесс согласования не поддерживается.
6. Один пользователь может иметь не более одного счёта одного типа в одной валюте.
7. Закрытый счёт не участвует в новых операциях.
8. Уход в минус допускается только для перевода, только для разрешённого типа счёта и только в пределах `negative\_balance\_limit`.
9. Пополнение и вывод не выполняются напрямую: они либо идут через заявку, либо недоступны в зависимости от режима работы системы.
10. Верхний уровень архитектуры делится по предметным контекстам, а не по ролям пользователей.

6. Допущения

1. Для каждой бизнес-операции используется единый словарь терминов и фиксированные перечисления статусов, ролей и типов объектов.
2. Все денежные операции предполагаются одновалютными в рамках одной операции, а валюты операции и счетов должны совпадать.
3. Ручная обработка заявок выполняется одним оператором на этапе принятия решения, поскольку в системе отсутствует многоэтапное согласование.
4. Источником истины по текущему состоянию счёта является поле `balance`, а проводки служат для учёта, аудита, сверки и восстановления.
5. Все доменные сущности должны сохранять историю изменений через статусы, аудит или связанные записи, а не через физическое удаление.
6. Система ориентирована на минималистичную модульную архитектуру: доменные правила сосредоточены в предметных модулях, а общие UI-компоненты не должны содержать прикладной или доменной логики.